An Exact Algorithm for the Multiple Vehicle Pickup and Delivery Problem

Sanjay Adhikary

December 6, 2025

---

1 INTRODUCTION

In this project, I implement an exact optimization framework for the Pickup and Delivery Problem (PDP) based on the integer programming formulation introduced by Lu and Dessouky (Lu & Dessouky 2004). Although the original work addresses the multiple-vehicle case, my implementation focuses on the core single-vehicle routing structure, including all essential components of the formulation: network flow constraints, pickup–delivery precedence, pairing constraints, implicit subtour elimination, and capacity consistency. Time-window constraints, which appear as an extension in the paper, are intentionally omitted.

To evaluate the behaviour of the model under different spatial patterns, I construct benchmark instances for both R-type (random) and C-type (clustered) datasets by generating pickup–delivery pairs, vehicle depots, and fixed-usage penalties when applicable. For the C-type datasets, I study two pairing conditions—pickup and delivery nodes located within the same cluster and located in different clusters—and additionally analyze a special case where each cluster contains exactly one pickup or delivery node to create a one-to-one cluster–customer structure.

2 PROBLEM FORMULATION

In this work, I implement the complete F1 formulation proposed by Lu and Dessouky, which includes all sixteen core constraints along with the full set of valid inequalities designed to strengthen the linear relaxation. The time-window extension of the model is intentionally omitted, and no experiments related to the PDPHTW are performed. The notation for node sets, parameters, and decision variables used in the formulation is summarised in Table 1. The full mixed-integer model corresponding to Formulation F1 is presented in Section 2.1, followed by the valid inequalities in Section 2.2. The overall workflow of the implemented branch-and-cut algorithm—used to solve the model on Solomon's R- and C-type datasets—is provided in the appendix (Figure 4).

Table 1: All the node sets, parameters and decision variables

Features	Components
Node sets	• $N_r^+ = \{1, \ldots, n\}$: set of pickup nodes. • $N_r^- = \{n+1, \ldots, 2n\}$: set of delivery nodes (node $n+i$ is delivery of request i). • $N_r = N_r^+ \cup N_r^- = \{1, \ldots, 2n\}$: all service nodes. • $N_q = \{2n+1, \ldots, 2n+m+1\}$: depot-related nodes (vehicle departure/return structure). • $N = N_r \cup N_q$: all nodes in the network. • $A \subseteq N \times N$: set of directed arcs (A = $A_r \cup A_q \cup A_{r,q}$. • A_c = Arcs between nodes of N_r • A_q = Arcs between nodes of N_q • $A_{r,q}$ = Arcs connecting nodes N_r & N_q
Parameters	• c_{ij}: cost of traveling from node i to node j (includes fixed vehicle cost on certain depot arcs if desired). • d_i: pickup demand of request i (for $i \in N_r^+$); delivery nodes inherit d_{i-n}. • D_v: capacity of vehicle v, $v = 1, \ldots, m$. • U_v: fixed cost of using vehicle v (can be embedded into c_{ij}).
Decision Variables	• $x_{ij} = \begin{cases} 1, & \text{if a vehicle travels directly from node } i \text{ to node } j, \\ 0, & \text{otherwise,} \end{cases}$ for all $(i,j) \in A$. • $b_{ij} = \begin{cases} 1, & \text{if node } i \text{ appears before node } j \text{ in the giant tour,} \\ 0, & \text{otherwise,} \end{cases}$ for all $i, j \in N$, $i \neq j$.

2.1 Mathematical Model (F1 formulation)

Objective function :

$$\min \sum_{i \in N} \sum_{j \in N} c_{ij} x_{ij}. \tag{1}$$

Flow Conservation (Degree Constraints)

Each service or depot node has exactly one successor and one predecessor:

$$\sum_{j \in N} x_{ij} = 1, \quad \forall i \in N, \tag{2}$$

$$\sum_{i \in N} x_{ij} = 1, \quad \forall j \in N. \tag{3}$$

Copy Constraints for b_{ij} (Idea Only)

These constraints enforce that if $x_{ij} = 1$ (i.e., j directly follows i), then $b_{kj} = b_{ki}$ for all k:

$$b_{ki} \leq b_{kj} + (1 - x_{ij}), \tag{4}$$

$$b_{kj} \leq b_{ki} + (1 - x_{ij}), \quad \forall (i,j) \in A,\ k \in N,\ k \neq i. \tag{5}$$

Routing–Ordering Consistency Constraint

$$x_{i,j} \leq b_{i,j} \quad \forall (i,j) \in A, \tag{6}$$

Self-Ordering

$$b_{ii} = 0, \quad \forall i \in N. \tag{7}$$

Pickup–Delivery Precedence

For each request $i \in N_r^+$:

$$\begin{aligned} b_{n+i,\,i} &= 0, \\ b_{i,\,n+i} &= 1. \end{aligned} \tag{8}$$

Pairing Constraints (Same Vehicle)

A simplified pairing constraint version:

$$b_{i,2n+j} = b_{n+i,2n+j}, \quad \forall i \in N_r^+,\ 2n + j \in N_q, \tag{9}$$

so pickup and delivery of request i lie within the same vehicle segment of the giant tour.

Capacity Constraints

g_i : Load change at node i and f_k is the capacity of vehicle k

$$g_j + \sum_{i \in N} b_{i,j} g_i \leq \sum_{k \in N_q} f_k b_{k,j} \quad \forall j \in N_r^+ \tag{10}$$

where q_j is the net pickup (+) or delivery (-) load at node j, and $D_{\max}$ is a vehicle capacity upper bound.

Start and End of Giant Tour

We fix the first and last nodes to specific depot nodes:

$$b_{i,\,2n+1} = 0, \quad \forall i \in N, \tag{11}$$
$$b_{i,\,2n+m+1} = 1, \quad \forall i \in N \setminus \{2n+m+1\}. \tag{12}$$

Binary Restrictions

$$x_{ij} \in \{0,1\}, \quad \forall (i,j) \in A, \tag{13}$$
$$b_{ij} \in \{0,1\}, \quad \forall i,j \in N,\ i \neq j. \tag{14}$$

2.2 Valid Inequalities

Transfer prior constraint

$$b_{n+i,h_1} + \sum_{j=1}^{k-1} b_{h_j,h_{j+1}} + b_{h_k,i} \ \leq \ k, \quad \forall\, i \in N_r^+,\ \forall\, (h_1,\ldots,h_k) \subset N \setminus \{i, n+i\},\ 1 \leq k \leq |N|-2. \tag{15}$$

Adjacent prior constraint

$$b_{k,i} + b_{k,i+n} \ \geq \ x_{i,k} + x_{k,i}, \qquad \forall\, i \in N_r^+,\ \forall\, k \in N \setminus \{i, i+n\}, \tag{16}$$
$$b_{i,k} + b_{i+n,k} \ \geq \ x_{i+n,k} + x_{k,i+n}, \qquad \forall\, i \in N_r^+,\ \forall\, k \in N \setminus \{i, i+n\}. \tag{17}$$

Pairing prior constraint

$$\sum_{k=1}^{2n+m+1} b_{k,i} + 1 \ \leq \ \sum_{k=1}^{2n+m+1} b_{k,i+n}, \quad \forall\, i \in N_r^+. \tag{18}$$

Vehicle return and zero load on returned vehicle constraint

$$\sum_{k=1}^{2n+m+1} b_{k,i}\, g_k \ = \ 0, \quad \forall\, i \in N_q. \tag{19}$$

$$b_{i,j} + b_{j,i} \ = \ 1, \quad \forall\, i,j \in N,\ i \neq j. \tag{20}$$

3 EXAMPLES

To understand the network structure and constraints we have created the toy example shown in the figure 1.

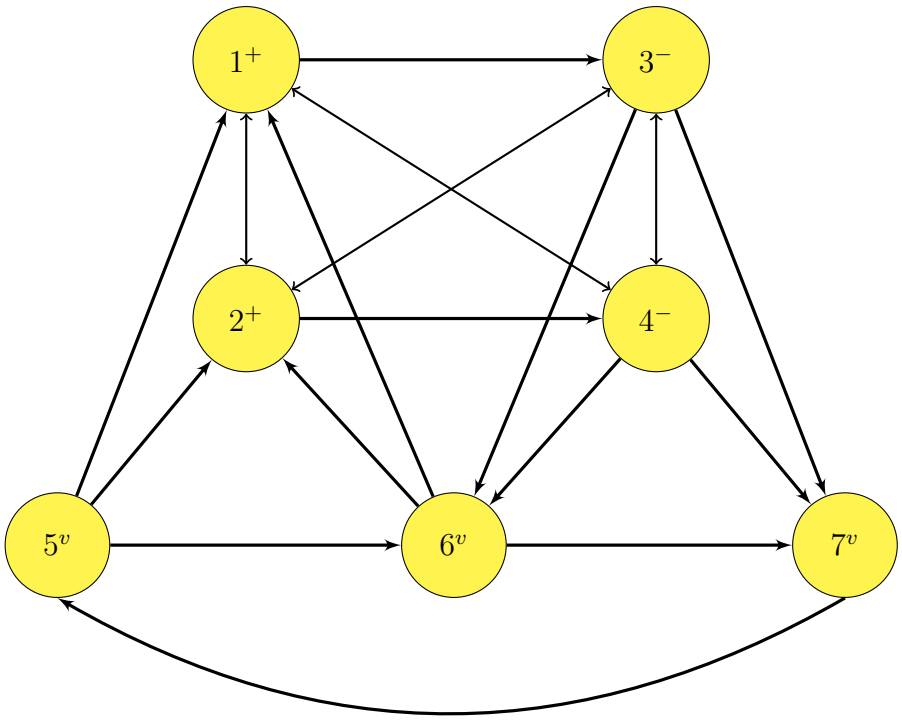


Figure 1: Network for a two customer and two vehicle case

3.1 Network construction

For n customer problem , each customer has a pickup node and a delivery request. Node i represents the pickup node and node n+i represents the delivery node for customer i. Now, if we have to route and scheduled m vehicles- 2n + 1 will be the departure depot of the first vehicle and 2n + m +1 will be the return depot of the last vehicle.
Now from the above figure 1 for 2 customer 2 vehicle problem , node 1 and 2 pickup node of customer 3 and 4 respectively. Whereas, node 5 is the departure depot of vehicle 1, node 6 is the departure depot of vehicle 2 as well as return depot of vehicle 1 and node 7 is the return depot of vehicle 2.

3.2 Avoiding pickup before delivery

Constraint 8 enforces that $b_{n+i,i} = 0$ and $b_{i,n+i} = 1$ i.e. pick up before delivery of the corresponding customer. Now, from the figure 1 any tour going from 3 to 1 or 4 to 2 is not allowed. Thus, we wont see any tour like 5-2-4-3-1-6 and that's why arcs from pickup to its corresponding delivery node is directed.

3.3 Pickup and delivery through the same vehicle

Constraint 9 enforces pickup and delivery of a customer through the same vehicle i.e. $b_{i,2n+j} = b_{n+i,2n+j}$. For our toy network (refer 1), we have 2 customer 2 vehicle problem , say i = 1 and j = 2 then if $b_{1,6} = 1$ then $b_{3,6}$ also has to be 1 i.e. if 1 is before 6 then 6 also has to be before 6. Thus the vehicle can not return to the depot without delivering.

4 DEMONSTRATION

4.1 Basic model formulation with and without valid inequalities

In this experiment, I implement the full F1 formulation exactly as described in the paper, first constructing the integer programming model containing all fundamental constraints of the pickup–delivery structure. I then evaluate the impact of incorporating the valid inequalities proposed by Lu and Dessouky—namely, the transfer prior, adjacent prior, pairing prior, and vehicle return constraints. The model is solved under two settings: (i) the baseline formulation without any valid inequalities, and (ii) the strengthened formulation with all valid inequalities included. A comparison of the computational performance and objective values for both settings is presented in Table 2.

Table 2: Computational comparison between using valid inequalities and without using valid inequalities

Num of cust	Without using valid inequalities				With using valid inequalities			
	Z_{LP}	Z_{OPT}	GAP(%)	CPU time (sec)	Z_{LP}	Z_{OPT}	GAP(%)	CPU time (sec)
5	226.8	264	14.08	0.71	226.8	264	14.08	0.5
8	243.3	249	2.29	0.88	243.3	249.0	2.29	1.0
10	312.5	354	11.72	23.82	312.5	354	11.72	22.81
12	259.6	330	21.34	1105.65	259.6	330.0	21.34	907.63

4.2 Implementation of the Main Branch-and-Cut Algorithm

Because the full F1 formulation contains a large number of constraints—many of which become redundant depending on the values of the decision variables—the original authors propose a dedicated branch-and-cut algorithm to improve computational efficiency. Following this approach, I implement the same framework to evaluate how the iterative constraint-generation mechanism affects performance. In particular, the number of constraints added during the solution process differs significantly between the basic "all-constraints-at-once" formulation and the iterative branch-and-cut version, as reported in the appendix (Table 6).

The algorithm also incorporates the prediction heuristic described in the paper, which attempts to construct a feasible integer solution from fractional LP solutions

at branching nodes. This heuristic helps reduce the size of the search tree. The effect is evident in the comparison of the number of explored nodes with and without prediction, also shown in the appendix.

During branching, a time limit of one hour is imposed, and the best feasible solution obtained within that time is reported. For each experimental setting, the results represent the average over five randomly generated samples.

After verifying the algorithmic behaviour, I apply the branch-and-cut framework to Solomon's R-type and C-type datasets and summarise the results. The scenarios considered are as follows:

- **Solomon's R-type datasets:** Customer locations are randomly distributed (see Figure B).
  - **Case I:** Zero vehicle usage cost, a common single depot (node 1 of the dataset) for all vehicles, fleet size set to 5, and vehicle capacity fixed at 200.
- **Solomon's C-type datasets:** Customer locations are spatially clustered (see Figure B).
  - Pickup and delivery locations restricted to lie within the same cluster.
  - Pickup and delivery locations forced to lie in different clusters.
  - No restrictions on pickup–delivery pairing (equivalent to the structure of the R-type dataset).

4.2.1 Computational results comparison of randomly generated customer nodes

Here we have compared total run time (sec) and GAP across those 4 datasets as shown in the figure 2 and 3 also i have attached the detailed result in a tabulated form in the appendix section (refer 3).

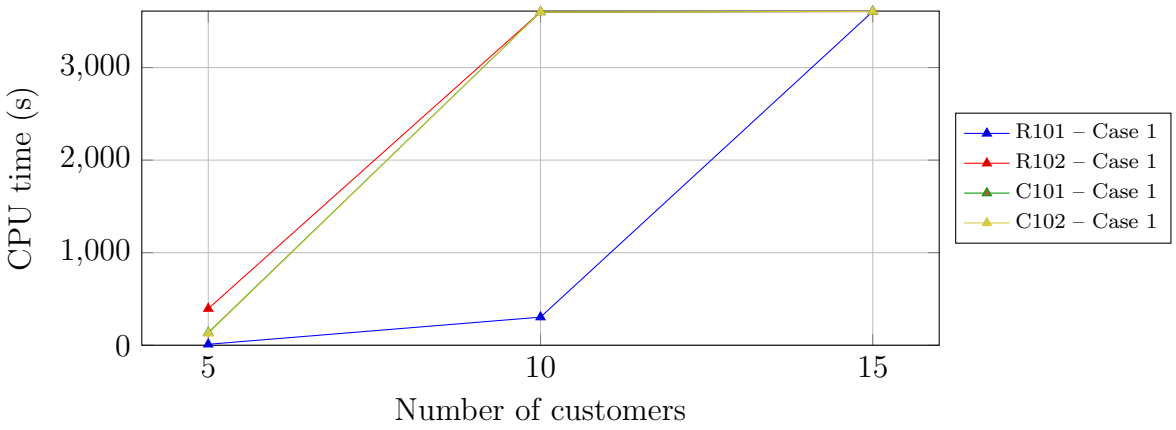


Figure 2: CPU time as a function of number of customers for different datasets and cases (dummy values).

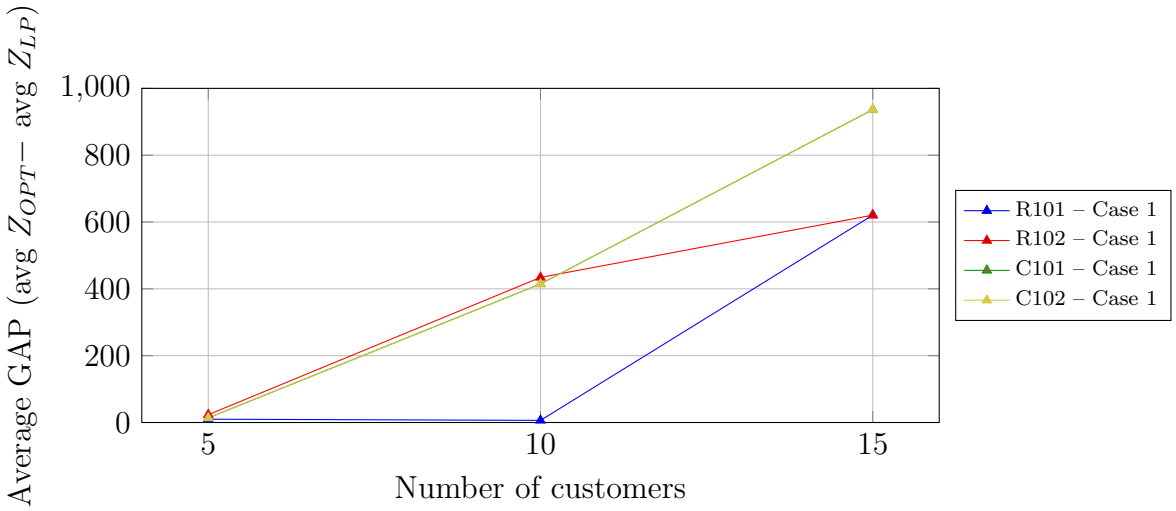


Figure 3: GAP as a function of number of customers for different datasets and cases (dummy values).

4.2.2 Computational results comparison of clustered customer nodes

For this the values are provided in a tabular form and attached in the appendix (refer 4 and 5). From the table it is obvious that with the increase in nos of customers computational complexity increases thus within the stipulated 1hr time it shown more gap for both pickup and delivery within the same and different cluster case. Among them also for pickup and delivery in different cluster takes longer time for same customer instances than within the same cluster case.

4.3 EXTENSIONS

Several meaningful extensions can be incorporated into the current F1-based implementation. A first extension is the inclusion of hard time windows at depots, pickup nodes, and delivery nodes. This requires adding service-time variables and temporal consistency constraints, allowing the model to handle scheduling-sensitive environments. The branch-and-cut framework remains applicable with minimal modification.

A second extension is to allow multiple customers to share the same pickup location, as in warehouse or hub-based systems. This can be handled by treating each request as a separate pickup–delivery pair while assigning the same pickup node, with only minor adjustments needed to avoid symmetry.

A third extension is to model customers with multiple pickup points before a single delivery. This is accommodated by linking several pickup nodes to one delivery node and enforcing precedence between them. The capacity and pairing structure of F1 naturally supports this with slight constraint extensions.

These modifications broaden the applicability of the formulation while preserving its overall structure and solvability.

References

1. Lu, Q. & Dessouky, M. An exact algorithm for the multiple vehicle pickup and delivery problem. *Transportation Science* **38,** 503–514 (2004).

A Demonstration results

A.1 For randomly generated pickup and delivery nodes

A.1.1 Case-1 : Used zero vehicle cost, single vehicle depot

Table 3: Computational results for randomly generated customer instances

Dataset	Number of customers	Average Z_{LP}	Average Z_{OPT}	CPU (sec)	Average vehicles used	Average gap ($Z_{OPT} - Z_{LP}$
R101	5	185.48	196.4	10.30	1	9.92
	10	272.42	278.80	303.68	1	6.4
	15	305.01	925.6	3605.48	1.8	620.59
R102	5	189.12	212.4	395.53	1	23.28
	10	278.33	712.6	3600.82	2.80	434.27
	15	305	925.6	3605	1.8	620.59
C101	5	243.73	257.40	136	1	13.67
	10	305.7	720.40	3600.85	2.20	414.7
	15	354.55	1291	3605.74	1.8	936.45
C102	5	243.73	257.40	128.14	1	13.67
	10	305.7	720.4	3600.67	2.2	414.7
	15	354.55	1291	3605.73	1.8	936.45

A.2 for clustered pickup and delivery nodes

A.2.1 Case-I: Pickup and delivery within the same cluster

Table 4: for C101 dataset comparison results between multiple customer from single cluster and single customer per cluster case

Dataset	Number of customers	Average Z_{LP}	Average Z_{OPT}	CPU (sec)	Average vehicles used	Average gap $(Z_{OPT} - Z_{LP}$
One customer per cluster	5	250.95	255	1.15	1	4.05
	10	333	339.4	20.6	1	6.4
	15	368.96	404.2	1429.51	1.2	35.24
Multiple customer from same cluster	5	232.6	235.4	2.89	1	2.8
	10	275.47	284	1542.32	1	8.53
	15	338.28	528.40	2794.97	1.60	190.12

A.2.2 Case-I: Pickup and delivery in different cluster

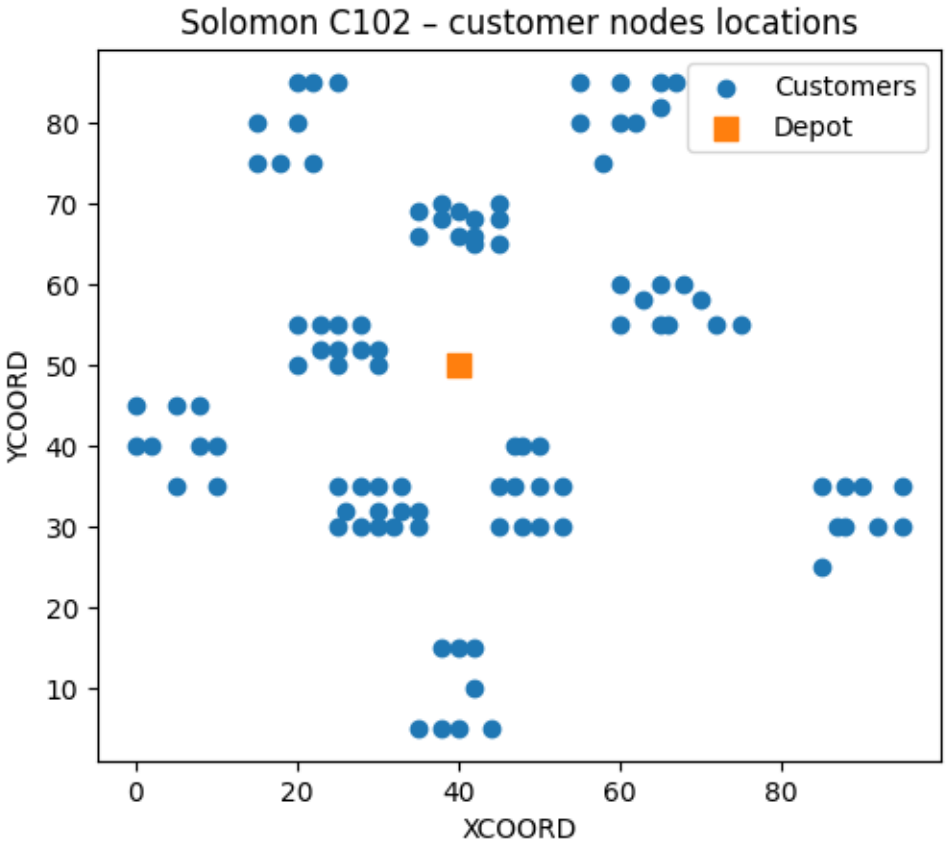
Table 5: for C101 dataset comparison results between multiple customer from single cluster and single customer per cluster case

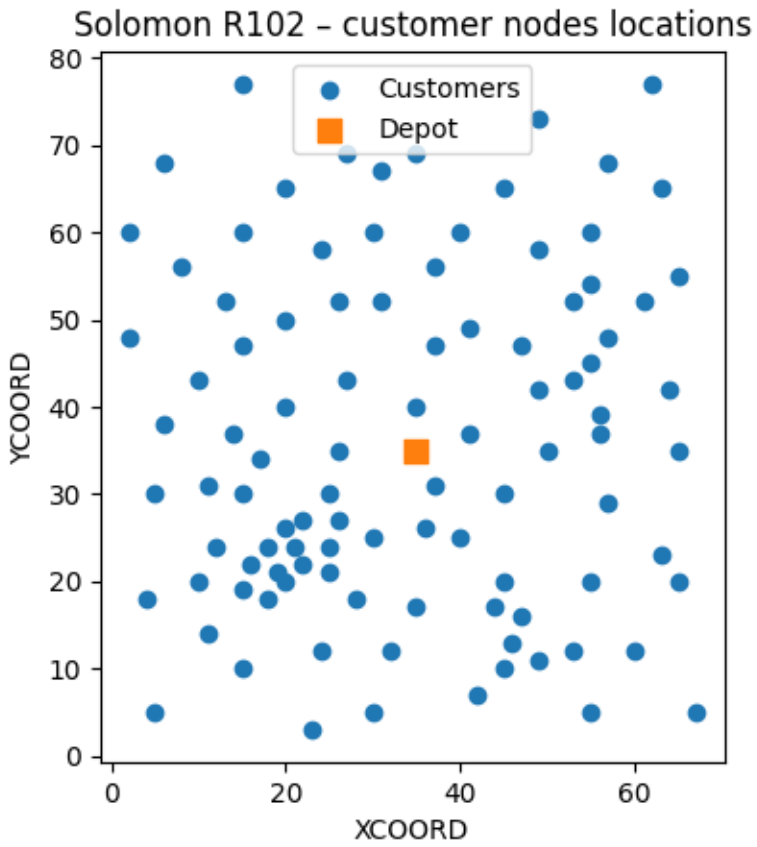
Dataset	Number of customers	Average Z_{LP}	Average Z_{OPT}	CPU (sec)	Average vehicles used	Average gap $(Z_{OPT} - Z_{LP}$
One customer per cluster	5	226.4	282.8	847.8	1.2	56.40
	10	318.2	868.6	3600.57	1.80	550.40
	15	341.92	1333.25	3600.77	1.75	991.33
Multiple customer from same cluster	5	224.25	240.25	48.50	1	16
	10					
	15					

B Dataset visualization

Table 6: Computational Comparison Between All Constraints from (3) to(5) in the Formulation (Adding Them Iteratively)

Nos of customers	CPU time for solving entire LP problem (sec)	Total nos of constraints	CPU time for solving the LP problem iteratively	Number of constraints add (3-5)
20	1.84	131220	1.65	314
40	22.01	1036840	11.03	760
60	127.54	3484860	29.17	1019
80	308.19	8243280	59.67	1191
100	492.87	16080100	129.24	1583




Solomon R102 – customer nodes locations
Customers
Depot
YCOORD
XCOORD
0
10
20
30
40
50
60
70
80
0
20
40
60

C Main Branch and Cut algorithm formulation

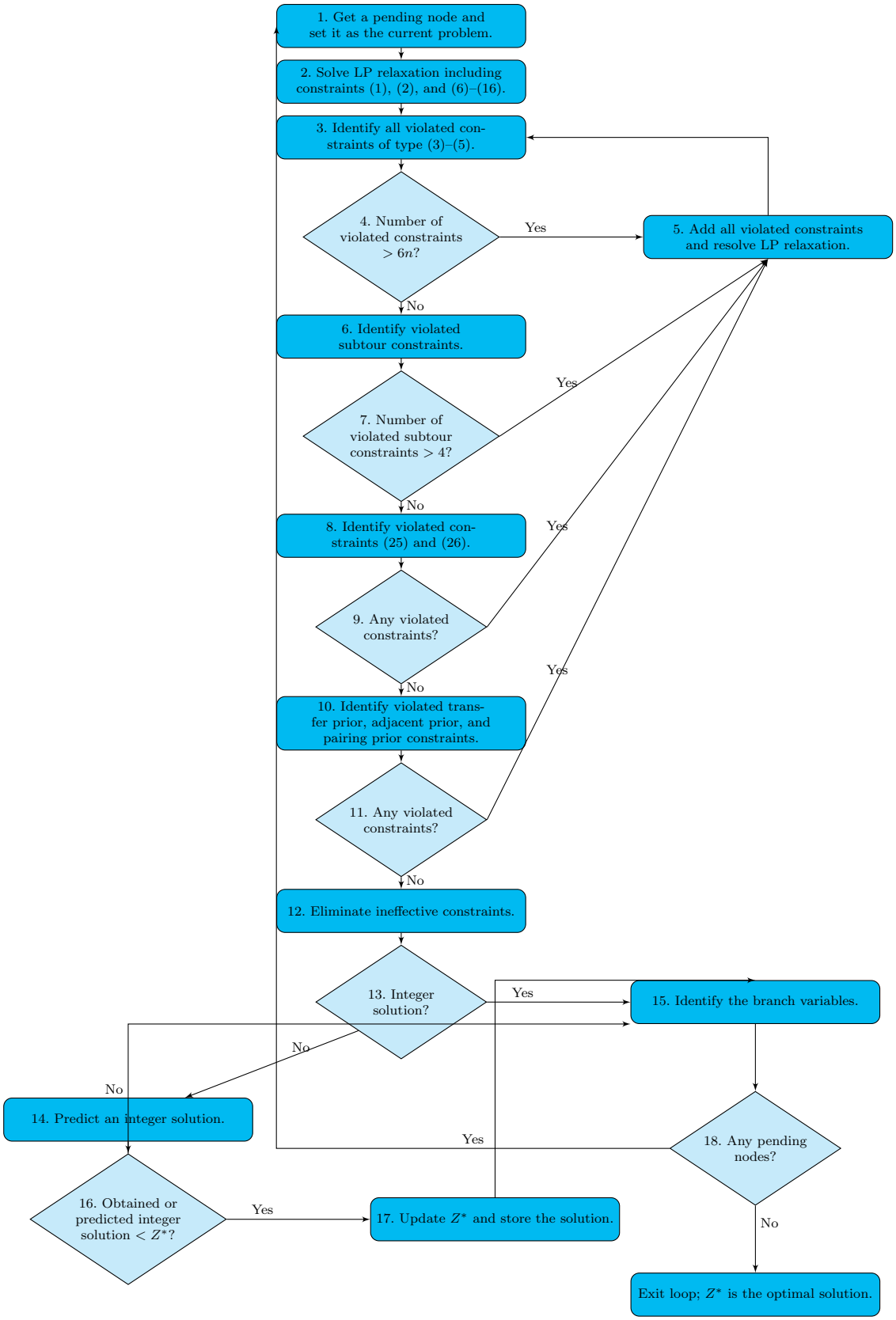


Figure 4: Branch-and-cut algorithm for the MVPDP